

“Universal Tickets - Universal Ticket Booking System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

V. HEMANTH :2520030025

K. CHAITANYA :2520030266

CH. VIVEK :2520030532

Under the Esteemed Guidance of

Dr. Prasanthi

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

Section: 9 | Team: 2 | Academic Year: 2026-2027

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY



Declaration

The Project Report entitled “Universal Tickets - Universal Ticket Booking System” is a record of Bonafide work of V. HEMANTH - 2520030025, K. CHAITANYA - 2520030266, CH. VIVEK - 2520030532 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering at K L University. The work presented in this report describes the design, database implementation, service integration, security mechanisms, testing, and deployment of the Universal Tickets system developed for the course Database Systems Engineering and Distributed Backend Development (25CS1302E). The results embodied in this report have not been copied from any other department, university, or institute.

V. HEMANTH - 2520030025
K. CHAITANYA - 2520030266
CH. VIVEK - 2520030532

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**



CERTIFICATE

This is certify that the project based report entitled “Universal Tickets - Universal Ticket Booking System” is a bonafide work done and submitted by V. HEMANTH - 2520030025, K. CHAITANYA - 2520030266, CH. VIVEK - 2520030532 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of Universal Tickets - Universal Ticket Booking System would not have been possible without the guidance, encouragement, and cooperation received throughout the development of the project. We sincerely thank all the faculty members and staff who supported us during the planning, implementation, testing, and documentation stages.

We express our deepest appreciation to our guide Dr. Prasanthi, Department of Computer Science and Engineering, for continuous guidance, technical suggestions, and encouragement during the development of the Universal Tickets system. The guidance helped us understand how database design, secure APIs, distributed backend services, and a responsive frontend can be integrated into one working application.

We also thank the faculty and department of K L (Deemed to be) University for providing the computing facilities, learning resources, laboratory support, and academic environment required to complete this course project for Database Systems Engineering and Distributed Backend Development (25CS1302E).

Finally, we thank our parents, friends, classmates, and team members for their support, feedback, and cooperation throughout the project development process.

V. HEMANTH - 2520030025

K. CHAITANYA - 2520030266

CH. VIVEK - 2520030532

Abstract

Universal Tickets is a full-stack Universal Ticket Booking System developed to provide a centralized digital platform for discovering events, checking availability, and completing ticket bookings. The project addresses the common problem of fragmented event information and manual booking workflows by providing a single responsive web application with role-aware access and a database-backed booking process.

The system combines a React 18 frontend with a Spring Boot backend, MySQL relational database, FastAPI gateway and search service, and a Node.js activity service. The architecture demonstrates polyglot backend development while preserving a clear separation of responsibilities. The frontend provides customer, organizer, verifier, and administrator interfaces; the Spring Boot service implements the main business logic and transactional booking workflow; the FastAPI service provides API aggregation and TF-IDF based semantic search; and the Node.js service records and aggregates activity information.

Security is implemented using BCrypt password hashing, JWT access and refresh tokens, authentication filters, and role-based access control. The project includes customer registration and login, organizer and verifier governance flows, event publishing, event discovery, booking creation, booking history, payment records, cancellation processing, seat restoration, and refund compensation logic. Booking operations are handled transactionally so that related database changes remain consistent.

The project was tested through frontend production builds, Spring Boot tests, FastAPI Pytest cases, Node.js activity-service smoke tests, application startup checks, authentication and RBAC checks, and responsive UI validation. The verified local audit reports successful execution of these core paths. The implementation also documents partial components for Redis, Kafka, and MongoDB activity aggregation, while clearly distinguishing them from live infrastructure and from a standalone vector database, which is not implemented in the current version.

Index

S. No.	Chapters	Topics	Page.no
	Acknowledgement		4
	Abstract		5
1	1 Introduction	1.1 Background of the project	5-8
		1.2 Problem statement	
		1.3 Scope of the project	
		1.4 Objective(s)	
		1.5 Importance of the application	
		1.6 Target users/audience	
2	2 System Requirements	2.1 Hardware requirements	9-10
		2.2 Software requirements	
		2.3 Development tools and frameworks	
3	3 Technology Stack	3.1 Front-end	11-13
		3.2 Back-end	
		3.3 Database	
		3.4 Version Control	
		3.5 APIs or External services	
4	4 System Architecture	4.1 High-level architecture diagram	14-15
		4.2 Description of each layer (frontend, backend, database)	
		4.3 Deployment architecture (local, Docker, CI/CD)	
5	5 Design	5.2 ER Diagram / Database schema	16-17
6	6 Implementation	6.1 Module-wise implementation	18-20
		6.2 Front-end logic (UI rendering, state management)	
		6.3 API integration	
		6.4 Authentication & Authorization	
7	7 Features	7.1 List of main features	21-23
		7.2 How each feature works (user perspective + technical description)	
8	8 Testing	8.1 Postman/API testing	24-26
		8.2 Integration testing	
		8.3 Tools used for testing	
		8.4 Test cases and results	
9	9 Deployment	9.1 Steps to deploy	27-28
		9.2 Environment configuration	
		9.3 Hosting of frontend and backend	
10	10 Challenges & Limitations	10.1 Issues faced during development	29-30
		10.2 Solutions applied	
		10.3 Current limitations or known bugs	
11	11 Future Enhancements	11.1 Planned features	31-32
		11.2 Possible integrations or optimizations	
12	12 Conclusion	12.1 Summary of the project	33-34
		12.2 What was achieved	
		12.3 Skills learned during development	

13	13 References	Books, tutorials, APIs, documentation sites used	35
14	14 Appendices	Screenshots of the app	36-40
		Selected command/code excerpts	
		Installation/setup instructions	
		User manual / demo flow	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Event ticketing has increasingly moved from physical counters and fragmented web pages to digital platforms. Customers expect to search for events, compare available choices, review event details, and reserve tickets through a single interface. Organizers, in turn, need a reliable way to create events, manage capacity, publish information, and monitor booking activity. The Universal Tickets project was conceived as a database-driven platform that connects these requirements in one system.

The project focuses on an integrated event-ticket workflow rather than a single static page. A customer can create an account, authenticate securely, browse published events, use search, select an event, create a booking, view booking history, and cancel a booking when allowed. Organizers can maintain event-related information through protected role-based interfaces, while verifiers and administrators provide governance and oversight.

From a database perspective, the system is centered on MySQL because customer identities, event records, venues, bookings, payments, and cancellations are highly structured and depend on referential integrity. The schema uses primary keys, foreign keys, constraints, transactional operations, indexes, joins, aggregate queries, CTEs, and window functions to demonstrate database engineering concepts required by the course.

The application is implemented as a polyglot full-stack solution. React and Vite are used for the presentation layer, Spring Boot and Spring Data JPA provide the main business service, FastAPI exposes a gateway and semantic-search layer, and Node.js with Express provides activity-service behavior. Docker and GitHub Actions support reproducible development and CI/CD workflows. This architecture also gives the team an academic opportunity to demonstrate SQL and NoSQL concepts, service boundaries, API security, and distributed-backend patterns.

The final project audit verified the core runtime path on a local Windows development environment. The verified stack includes React 18.2.0, Vite 8.2.2, Java 21.0.10, Spring Boot 3.2.3, MySQL 8.0.46, FastAPI 0.110.0, Pydantic 2.6.4, JWT 0.12.5, BCrypt 6.2.2, Node.js 20.x, Express

4.19.2, Docker 26.x, Docker Compose 2.x, GitHub Actions, OpenAPI/Swagger, and Spring Actuator.

1.2 Problem Statement

Many ticketing workflows separate discovery, authentication, booking, payment records, and cancellation management across disconnected screens or services. This separation can make it difficult for customers to understand booking status and for administrators to maintain consistent records. A system that does not enforce strong relationships between events, venues, users, and bookings can also create data inconsistency.

From an operational point of view, organizers need controlled access to event management functions, while customers should not be able to access administrative operations. Verifiers and administrators require separate governance capabilities. A centralized system therefore needs authentication, authorization, role-specific routing, and a database model that clearly defines ownership and relationships.

The project addresses these problems by building a unified ticketing system with a relational database, transaction-aware booking services, secure authentication, API integration, semantic search, activity tracking, and role-based dashboards. The solution is designed to demonstrate how a DBMS-focused application can be combined with modern distributed backend techniques without losing clarity of data ownership and business rules.

1.3 Scope of the Project

- Customer registration, login, JWT-based authentication, event discovery, search, booking, booking history, and cancellation.
- Organizer profile and event-management workflows, with verification and administrative governance.
- MySQL-based persistence for users, organizers, verifiers, venues, events, bookings, payments, and cancellations.
- FastAPI gateway and TF-IDF/cosine-similarity semantic search over available event information.
- Node.js activity service for activity aggregation and audit-oriented application information.

- Containerized local deployment, service health checks, CI/CD workflow, and project documentation.
- Advanced distributed components are documented according to their real implementation status; Redis, Kafka, and MongoDB-related features are marked PARTIAL where live infrastructure is not verified.

1.4 Objectives of the Project

1. Design a responsive web interface for centralized event-ticket discovery and booking.
2. Implement a normalized relational schema with strong keys, relationships, and transaction integrity.
3. Secure account access using BCrypt password hashing, JWT tokens, and RBAC.
4. Implement a booking and cancellation workflow that preserves seat and payment consistency.
5. Demonstrate service-oriented integration using Spring Boot, FastAPI, and Node.js.
6. Provide semantic event search using TF-IDF and cosine similarity.
7. Validate the solution using automated tests, smoke tests, API checks, and responsive UI validation.

1.5 Importance of the Application

Universal Tickets provides a single source of truth for the major entities involved in ticket booking. The relational model connects users to bookings, bookings to events, events to venues and organizers, and payments/cancellations to their corresponding bookings. This relationship-driven design reduces duplication and makes it possible to answer operational questions using SQL joins and aggregates.

The project is academically important because it combines database systems engineering with distributed backend development. The system demonstrates the practical use of ACID-oriented transaction management, API security, microservice boundaries, semantic search, Docker-based deployment, and CI/CD. These features map directly to the learning outcomes of the course while remaining grounded in a working application.

The application is also useful from a governance perspective. A customer, organizer, verifier, and administrator have different responsibilities. The role model prevents unauthorized operations and provides clear separation between customer-facing booking actions and management operations.

1.6 Target Users / Audience

- Customers: browse events, search, authenticate, book tickets, review booking history, and cancel eligible bookings.
- Organizers: manage organizer-specific event information and participate in the event publishing workflow.
- Verifiers: review organizer/event-related governance information where permitted.
- Administrators: view governance metrics, manage system-level functions, and access protected administrative routes.
- Project evaluators and faculty: inspect the database design, API architecture, implementation choices, test evidence, and course-outcome mapping.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project can be developed and demonstrated on a modern laptop or desktop with a multi-core CPU, at least 8 GB RAM, and sufficient SSD space for Java, Node.js, Python, MySQL, Docker images, browser instances, and source dependencies. For smoother simultaneous execution of Spring Boot, FastAPI, Node.js, MySQL, the React development server, and browser testing, 16 GB RAM is recommended.

The frontend is browser-based, so end users do not need specialized hardware. A modern desktop, laptop, tablet, or smartphone with a current browser is sufficient. Responsive validation was performed across widths from approximately 375 px to 1920 px during the final audit.

A stable network connection is useful during package installation, GitHub synchronization, Docker image retrieval, and API-based service testing. Local database operations and the core booking workflow can still be demonstrated without depending on external travel APIs.

2.2 Software Requirements

Software	Purpose	Verified Version / Status
Operating System	Development and demonstration	Windows local environment; Linux compatible
Java	Spring Boot backend	21.0.10
Spring Boot	Main business backend	3.2.3
React	Frontend UI	18.2.0
Vite	Frontend build/dev server	8.2.2
Python	FastAPI gateway runtime	3.x environment
FastAPI	Gateway / search service	0.110.0
Node.js	Activity service	20.x
Express	Node HTTP service	4.19.2
MySQL	Relational database	8.0.46
Docker / Compose	Containerization	26.x / 2.x
Git / GitHub	Source control and CI/CD	Used
Browser	UI testing	Chrome / Brave / other modern browser

2.3 Development Tools and Frameworks

Visual Studio Code and terminal tools were used for source editing, builds, debugging, and repository operations. Maven manages the Spring Boot backend build, while npm manages the React and Node.js services. Python packaging and Pytest are used for the FastAPI component.

Swagger/OpenAPI through FastAPI provides API documentation at the gateway layer. Spring Actuator provides health-oriented runtime endpoints for the Java service. Docker Compose provides a repeatable multi-service environment, while GitHub Actions automates build and test checks for repository changes.

Testing was performed using automated unit/integration checks, API-level smoke verification, browser-based UI validation, and role/access checks. The final audit recorded frontend build success, backend test success, FastAPI test success, Node activity-service smoke success, and successful Spring Boot/Tomcat startup.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend is implemented with React 18.2.0 and Vite 8.2.2. React provides reusable components and client-side routing, while Vite provides fast development and production-build tooling. The UI includes customer-facing event discovery, login and registration, protected role-based routes, booking pages, booking history, administrative/organizer interfaces, and informational pages.

The frontend maintains the authenticated user context needed to decide which navigation and routes are available. Defensive guards were added to prevent incomplete local-storage user data from causing render failures. Protected routes ensure that customers cannot navigate into administrative functionality, while the interface exposes role-appropriate menu items for organizers, verifiers, and administrators.

The UI supports light, dark, and system themes and was validated across a wide range of viewport widths. A final browser audit confirmed the root page, navbar, footer, static routes, console behavior, and responsive layout in a working browser environment.

3.2 Back-End Design

The main backend is implemented using Java 21 and Spring Boot 3.2.3. Spring Data JPA is used for persistence and Hibernate provides ORM behavior. Controllers expose REST endpoints, services implement business rules, and repositories interact with the MySQL database.

The backend contains authentication, authorization, event, booking, payment, cancellation, organizer, verifier, and administrator operations. JWT access and refresh tokens are used for session continuity. The access token is HMAC-SHA256 based and is configured for approximately 24 hours, while refresh tokens are configured for approximately 7 days in the verified implementation.

Booking and cancellation logic uses transactional processing. A successful booking creates related booking/payment records and updates seat-related state, while cancellation triggers compensation logic such as restoring availability and handling the associated refund state. This pattern illustrates transaction integrity and application-level Saga/compensation concepts.

3.3 Database Design

MySQL 8.0.46 is the primary relational database. The verified core schema contains the tables users, organizer_profiles, organizer_verifiers, venues, events, bookings, payments, and cancellations. The schema uses primary keys and foreign keys to maintain relationships and supports structured queries over users, events, venues, and transactional records.

The principal relationships include bookings.user_id -> users.user_id, bookings.event_id -> events.event_id, cancellations.booking_id -> bookings.booking_id, events.organizer_id -> organizer_profiles.organizer_id, events.venue_id -> venues.venue_id, payments.booking_id -> bookings.booking_id, venues.organizer_id -> organizer_profiles.organizer_id, and organizer_profiles.verified_by -> organizer_verifiers.verifier_id.

The system also demonstrates advanced SQL through joins, aggregate queries, CTEs, and window functions. Public-facing identifiers follow structured formats such as CUS-2026-000001, ORG-2026-000001, VEN-2026-000001, EVT-2026-000001, BOOK-2026-000001, PAY-2026-000001, and CAN-2026-000001 where applicable.

3.4 Version Control

Git and GitHub are used for source-code management. The project is maintained as one shared repository so that all members contribute to the same integrated application. Each team member works from an individual branch and creates meaningful commits using their own GitHub account. This provides traceability and prevents the project from being split into disconnected repositories.

A practical responsibility split assigns frontend and demo documentation to V. Hemanth, Spring Boot/MySQL/security/booking to K. Chaitanya, and FastAPI/Node/infrastructure/DevOps to CH. Vivek. The final repository is merged and validated as one system.

3.5 APIs or External Services

The frontend communicates with the backend and gateway using HTTP/JSON APIs. The FastAPI gateway exposes documented endpoints through OpenAPI/Swagger and performs search-oriented aggregation. The Node.js activity service exposes activity-related endpoints and can aggregate activity information in-memory when a live document store is not available.

The project contains code paths for Redis and Kafka concepts, but the final audit classifies them as PARTIAL because a live Redis daemon and a live Kafka broker/consumer were not verified.

Similarly, MongoDB document-store integration is PARTIAL and may use an in-memory fallback for the verified activity aggregation path. A standalone vector database such as Qdrant, Pinecone, Milvus, or pgvector is NOT implemented; the current semantic search uses TF-IDF vectors and cosine similarity.

Technology / Component	Role in Universal Tickets	Status
React + Vite	Responsive frontend and SPA routing	VERIFIED
Spring Boot + JPA	Core business backend	VERIFIED
MySQL	Transactional relational persistence	VERIFIED
FastAPI + Pydantic	Gateway and search API	VERIFIED
TF-IDF + cosine similarity	Semantic event search	VERIFIED
Node.js + Express	Activity service	VERIFIED
JWT + BCrypt	Authentication / password security	VERIFIED
Docker + Compose	Containerized environment	VERIFIED
GitHub Actions	CI/CD automation	VERIFIED
OpenAPI / Swagger	API documentation	VERIFIED
Spring Actuator	Backend health/monitoring endpoints	VERIFIED
MongoDB	Document/activity storage path	PARTIAL
Redis	Caching/integration component	PARTIAL
Kafka	Event-streaming producer path	PARTIAL
Standalone vector DB	Dedicated vector database	NOT IMPLEMENTED

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

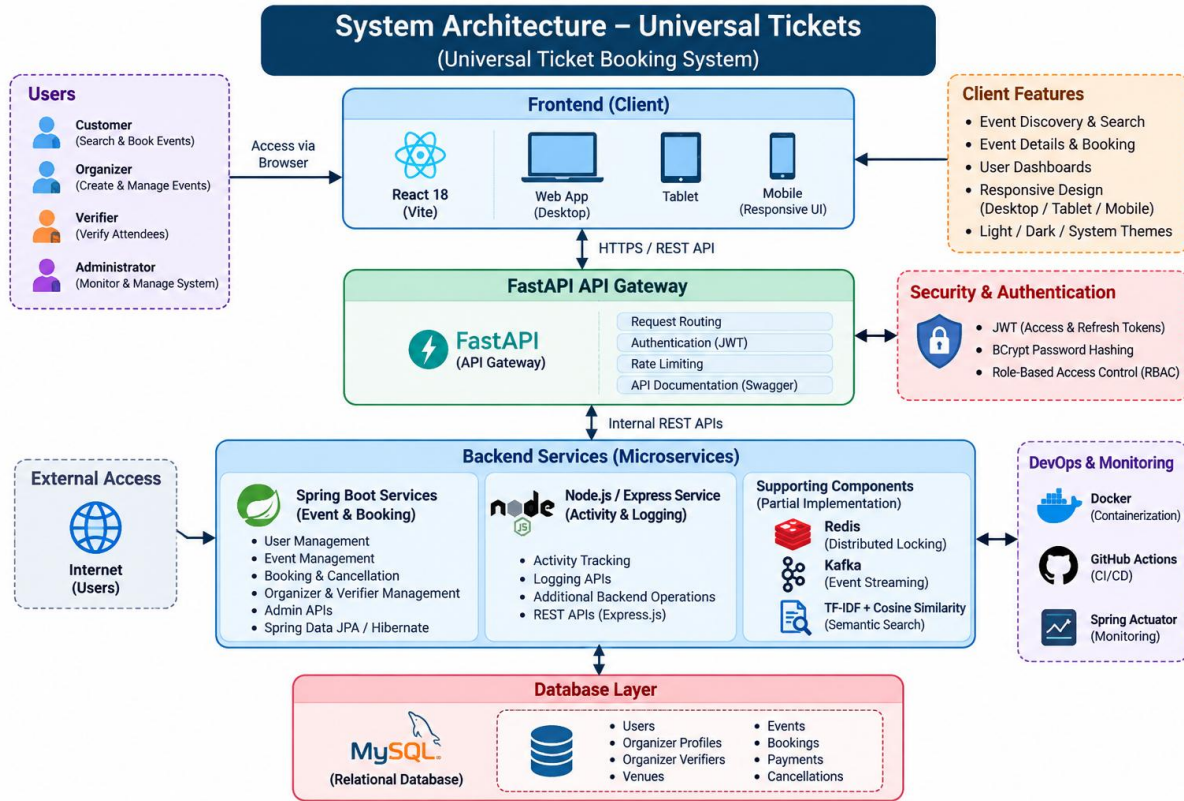


Figure 4.1: High-level architecture of Universal Tickets

4.2 Description of Each Layer (Frontend, Backend, Database)

The frontend layer is the user-facing React application. It collects user actions such as registration, login, event search, event selection, booking, and cancellation. The frontend does not directly manipulate the database. Instead, it invokes protected HTTP APIs and renders the responses in reusable components.

The Spring Boot layer is the main business-logic service. It validates authenticated requests, applies role rules, orchestrates booking and cancellation logic, and uses Spring Data JPA to persist entities in MySQL. The backend also hosts the administrative governance APIs and role-specific features.

The FastAPI layer acts as a gateway/search service. It exposes API documentation, supports semantic event search, and provides an additional backend boundary for distributed-service

coursework. Semantic search uses TF-IDF to represent text and cosine similarity to compare query and event representations.

The Node.js activity service provides a separate service boundary for activity logging and aggregation. Its verified path can operate with an in-memory fallback when an external document store is unavailable. This illustrates service decomposition while keeping the local demonstration reliable.

The database layer is anchored by MySQL. The schema stores structured system-of-record data such as users, organizers, venues, events, bookings, payments, and cancellations. Referential integrity and transactions ensure that booking changes remain consistent. The polyglot components are treated as supporting services rather than as replacements for the verified MySQL system of record.

4.3 Deployment Architecture (Local, Docker, CI/CD Pipelines)

During development and demonstration, the application can be started locally with the React frontend on port 5173, the Spring Boot backend on port 8080, the FastAPI gateway on port 8000, the Node.js activity service on port 5001, and MySQL on port 3306. This arrangement allows each service to be debugged independently while still functioning as one application.

Docker and Docker Compose provide an alternative containerized environment. The objective is to keep service configuration reproducible across machines and to reduce differences caused by local dependency installation. The repository includes scripts and compose configuration for service startup and project checks.

GitHub Actions provides CI/CD automation for repository validation. The verified pipeline focuses on building and testing project components. The report intentionally does not claim a production cloud hosting deployment, because the final audit verified the project locally and through container/CI tooling rather than through a live public hosting environment.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database Schema

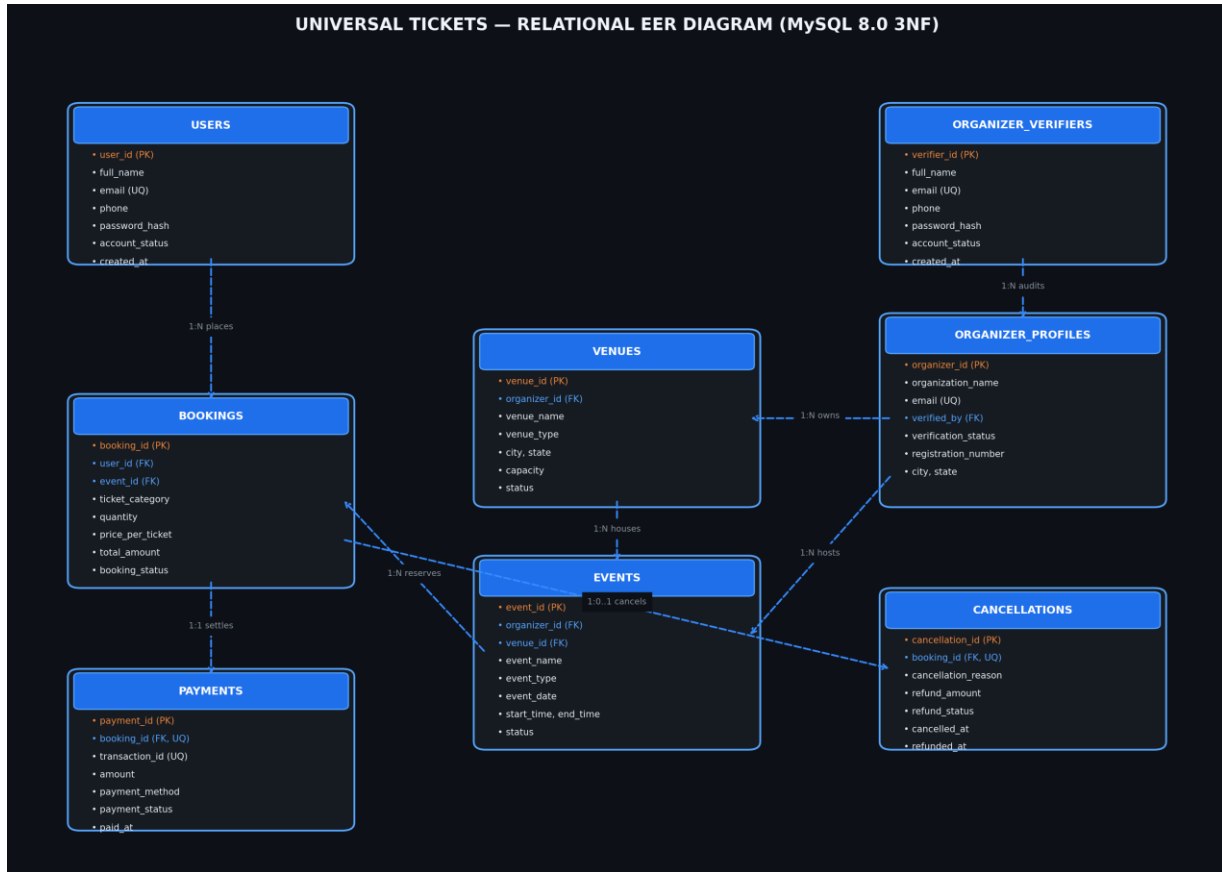


Figure 5.1: Universal Tickets relational ER diagram

Table	Key Fields	Purpose
users	user_id, email, password_hash, role	Customer and identity records
organizer_profiles	organizer_id, user/reference data, verified_by	Organizer information and verification link
organizer_verifiers	verifier_id, verifier details	Verification/governance records
venues	venue_id, organizer_id, venue details	Event location and organizer ownership
events	event_id, organizer_id, venue_id, event details	Published and managed events
bookings	booking_id, user_id, event_id, booking details	Customer ticket reservations
payments	payment_id, booking_id, amount, status	Payment record for a booking
cancellations	cancellation_id, booking_id, refund/status details	Cancellation and compensation record

The design emphasizes normalized ownership and traceable relationships. A customer can have multiple bookings, an event belongs to an organizer and a venue, and a booking can have related

payment and cancellation records. This structure supports transaction-oriented processing and enables administrative reporting without duplicating core facts across tables.

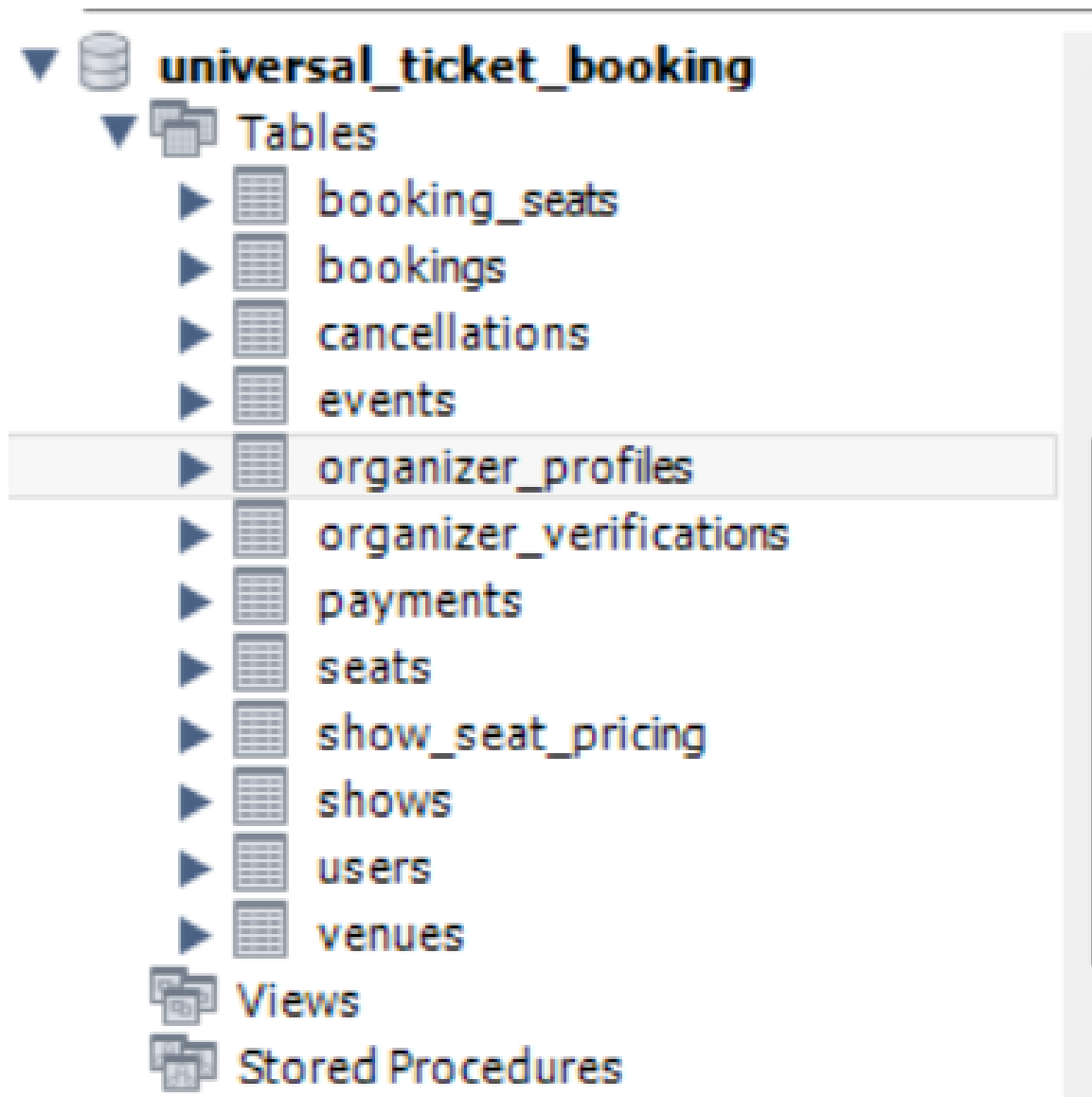


Figure 5.2: Representative database tables used in the project

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

User & Authentication Module: Registration, login, password verification using BCrypt, JWT access/refresh token lifecycle, authenticated session state, and role assignment.

Customer Module: Event browsing, search, event details, booking, booking history, cancellation, and customer-specific protected routes.

Organizer Module: Organizer profile handling, event management, event publishing workflow, and organizer-specific views.

Verifier Module: Verifier-facing governance functions for organizer/event review where allowed.

Admin Module: Administrative overview, counts, governance controls, and protected admin endpoints.

Booking Module: Transactional booking creation, related payment records, seat/state updates, and booking status management.

Cancellation Module: Eligibility checks, cancellation update, seat restoration, and refund compensation state.

Search Module: FastAPI semantic search using TF-IDF feature vectors and cosine similarity.

Activity Module: Node.js activity logging/aggregation with verified in-memory fallback behavior.

Infrastructure Module: Docker Compose, environment configuration, health checks, GitHub Actions, and startup scripts.

6.2 Front-end Logic (UI Rendering, State Management)

The React application is organized around reusable pages and components. Navigation elements are rendered according to user authentication state and role. Protected routes prevent direct access to management pages from an unauthorized customer session. User context and token information are maintained on the client side so that authenticated requests can be made consistently.

Defensive rendering was an important implementation concern. Earlier test iterations exposed cases where incomplete localStorage data could cause calls such as name splitting or role normalization to fail. The final code uses guards so that the application renders safely even when stored user information is incomplete or missing.

The frontend also contains search and route handling improvements. Static pages such as admin, about, help, contact, terms, privacy, and refunds were explicitly routed so the browser does not fall through to an unintended blank state.

6.3 API Integration

The frontend communicates with REST endpoints over JSON. Authentication requests establish the JWT session, after which protected endpoints carry the access token in the request. The gateway and activity service are separated into their own ports so that service boundaries are visible during demonstrations and testing.

The final service map is: React/Vite frontend - 5173; Spring Boot backend - 8080; FastAPI gateway/search - 8000; Node.js activity service - 5001; MySQL - 3306. This separation makes it possible to validate each service independently and also to demonstrate integrated behavior.

The FastAPI gateway provides OpenAPI documentation and the search layer. The Node activity service provides a dedicated service boundary for audit-oriented data. The architecture keeps the transactional system of record in MySQL while exposing supporting services where the course requirements benefit from polyglot patterns.

6.4 Authentication & Authorization

During login, the backend verifies the submitted password against the BCrypt hash stored for the account. Successful authentication results in JWT access and refresh tokens. Invalid credentials do not create a new account and do not fall back to a default password.

Authorization is role-based. Customer, organizer, verifier, and administrator permissions are enforced on protected routes. The verified audit includes a customer request to an admin endpoint returning HTTP 403 and an unauthenticated protected request being rejected.

Environment-backed configuration is used for the database connection and JWT signing secret. Local demonstration credentials exist for the different roles, but passwords are kept out of shareable repository documentation and should be stored separately when the system is distributed.

CHAPTER -7 FEATURES

7.1 List of Main Features

- Customer registration and secure login.
- BCrypt password verification and JWT access/refresh tokens.
- Role-based access control for customer, organizer, verifier, and admin.
- Event creation/publishing workflow for authorized organizers.
- Event discovery and semantic search.
- Booking creation with transactional consistency.
- Booking history for customers.
- Cancellation handling with seat restoration and refund compensation state.
- Administrator governance dashboard and live overview counts.
- Responsive interface and Light / Dark / System themes.
- FastAPI Swagger/OpenAPI documentation.
- Node.js activity service and aggregation path.
- Docker/Compose and GitHub Actions support.
- Actuator health/runtime support for the Spring Boot service.

7.2 How Each Feature Works (User Perspective + Technical Description)

Registration and Login: Customers register an account and log in from the frontend. The backend validates credentials using BCrypt and returns JWT tokens on success. The frontend stores the authenticated state and exposes customer-appropriate navigation.

Role-based Navigation: Users see only the pages intended for their role. Backend endpoints also enforce the role so a user cannot bypass the UI by directly calling a protected API.

Event Discovery: Published event records are loaded from the backend. The customer can inspect event details, venue information, organizer information where exposed, availability-related information, and booking actions.

Semantic Search: The FastAPI layer converts event text and the user query into TF-IDF representations and compares them using cosine similarity. Results are therefore ranked by textual similarity rather than by an external vector database.

Booking: A booking request is validated and handled inside a transaction. Related booking and payment data are created and the event availability state is updated consistently.

Cancellation: An eligible cancellation updates the cancellation record and compensates the booking state. Seat availability is restored and refund status is handled as part of the business flow.

Admin Governance: An authenticated administrator can open the admin portal and view system-level overview information such as user/event/booking statistics. Non-admin requests are blocked by authorization checks.

Activity Service: Activity information is handled by a separate Express service. The current verified implementation can aggregate information without requiring a live MongoDB server, while keeping the design ready for a document-store integration.

Theme and Responsiveness: The interface supports light, dark, and system appearance modes and adapts to mobile and desktop widths. Responsive validation covered approximately 375 px to 1920 px.

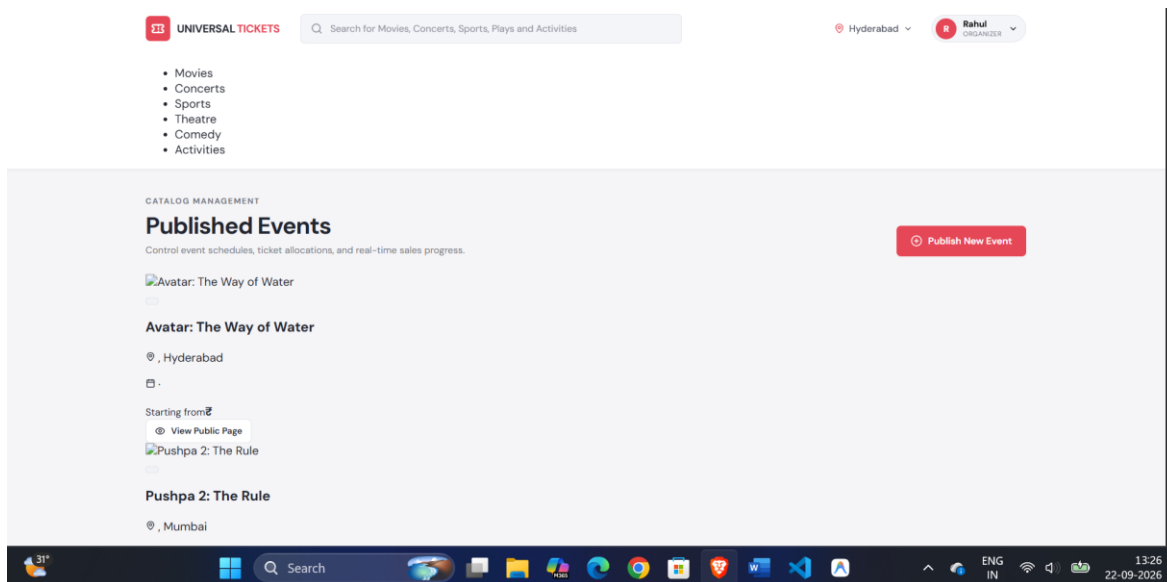


Figure 7.1: Event publishing / published-events view

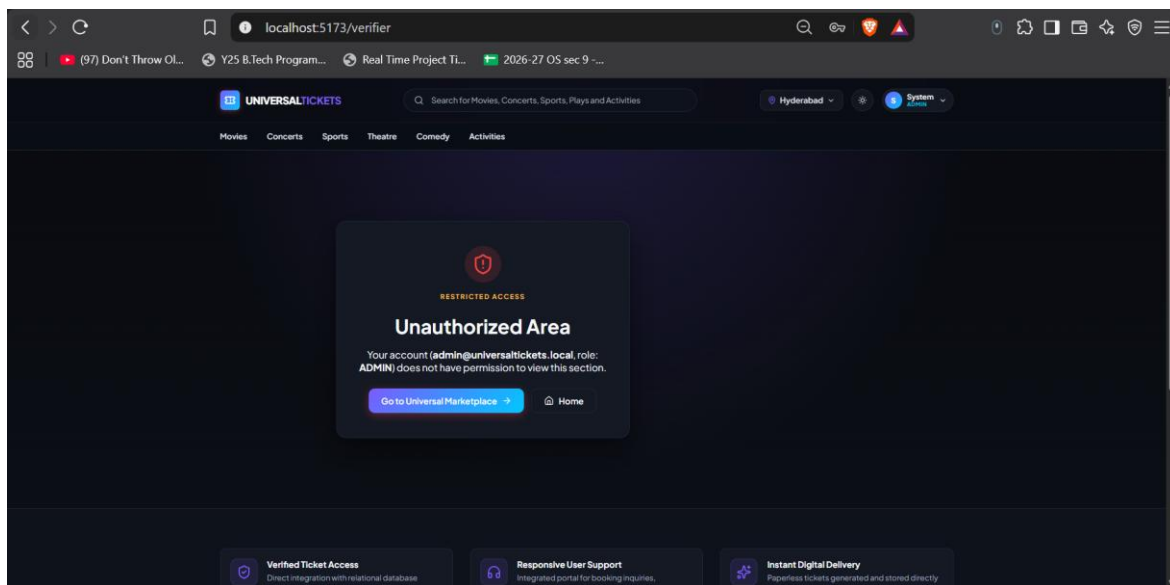


Figure 7.2: Unauthorized access response demonstrating RBAC enforcement

CHAPTER -8 TESTING

8.1 API Testing (Frontend / Backend)

API verification was carried out by invoking authentication, event, booking, cancellation, and role-protected endpoints. The tests checked both valid and invalid requests, including missing authentication and incorrect role access. The final audit confirmed that the key protected paths behaved according to their expected permissions.

The FastAPI service was tested with Pytest. The verified result was 4 out of 4 tests passing. The Spring Boot backend test suite reported 3 out of 3 tests passing. The Node.js activity service also completed its smoke test successfully.

The frontend was tested through a production build and browser validation. The final build succeeded, and the application was visually checked for navbar, footer, routes, console behavior, and responsive layout.

8.2 Integration Testing

Integration testing focused on the complete path from user interface to backend and database, with supporting checks for the FastAPI and Node services. The objective was to ensure that a customer could authenticate, discover an event, create a booking, see the booking, and perform an eligible cancellation without breaking database consistency.

Application startup was also verified. A Windows-specific Tomcat loopback startup issue was encountered during development and resolved using an `Http11Nio2Protocol` customizer. After the fix, the Spring Boot application started successfully and served the verified endpoints.

The test process also included database verification and live-data inspection. The verified local dataset reported 43 customers, 11 organizers, 30 events, and 52 bookings during the final audit snapshot.

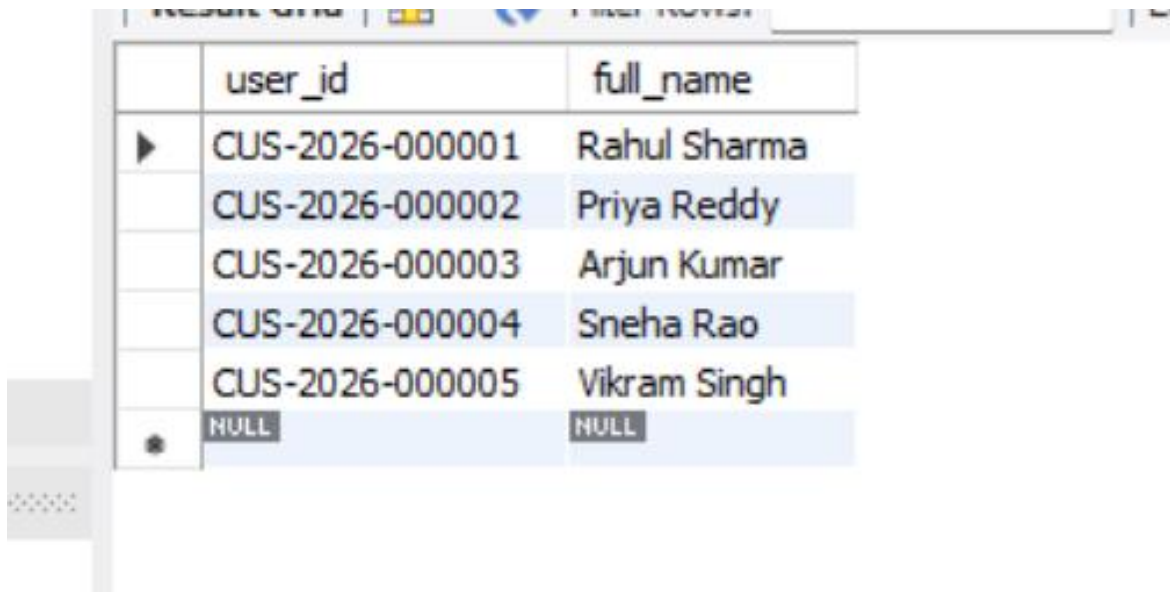
8.3 Tools Used for Testing

- Maven test lifecycle for Spring Boot tests.
- Pytest for FastAPI gateway/search checks.

- Node.js smoke-test scripts for the activity service.
- Browser developer tools and headless/browser validation for frontend rendering.
- MySQL queries and database inspection for relational data validation.
- Docker / Compose health and startup checks.
- Spring Actuator for runtime/health verification.

8.4 Test Cases and Results

ID	Test	Expected / Observed Result	Status
TC-01	Customer registration	Valid details create account	PASS
TC-02	Valid login	BCrypt verified; JWT issued	PASS
TC-03	Invalid login	Request rejected; no auto-provisioning	PASS
TC-04	Customer -> admin endpoint	HTTP 403 / blocked	PASS
TC-05	Unauthenticated protected request	Request rejected	PASS
TC-06	Event discovery	Published events load	PASS
TC-07	Semantic search	Relevant ranked results returned	PASS
TC-08	Booking transaction	Booking/payment state consistent	PASS
TC-09	Cancellation	Seat restoration/refund compensation	PASS
TC-10	Spring Boot tests	3/3 passing	PASS
TC-11	FastAPI tests	4/4 passing	PASS
TC-12	Node activity smoke test	Service path responds correctly	PASS
TC-13	Frontend build	Production build succeeds	PASS
TC-14	Responsive UI	375-1920 px validation	PASS
TC-15	Spring startup	Tomcat/application starts successfully	PASS



	user_id	full_name
▶	CUS-2026-000001	Rahul Sharma
	CUS-2026-000002	Priya Reddy
	CUS-2026-000003	Arjun Kumar
	CUS-2026-000004	Sneha Rao
	CUS-2026-000005	Vikram Singh
•	NULL	NULL

Figure 8.1: Users table verification snapshot

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

1. Install Java 21, Node.js 20.x, Python 3.x, MySQL 8.x, and Docker if a containerized run is required.
2. Clone the single GitHub repository containing frontend, backend, FastAPI gateway, activity service, database scripts, and documentation.
3. Create local environment variables for database credentials and JWT signing configuration. Do not commit real secrets to GitHub.
4. Create the MySQL database schema and verify connectivity from the Spring Boot backend.
5. Start the Spring Boot backend on port 8080.
6. Start the FastAPI gateway/search service on port 8000.
7. Start the Node.js activity service on port 5001.
8. Start the React/Vite frontend on port 5173 for development or build the production bundle for a static hosting environment.
9. Run automated tests and service health checks before the demo or release.
10. For a containerized run, use Docker Compose and verify the service network plus database connectivity.

9.2 Environment Configuration

Environment configuration separates application code from machine-specific settings. Database host, port, name, username, password, and JWT signing secret are supplied through environment variables or local configuration. This prevents secrets from being hard-coded in the source and makes the same application easier to run across development machines.

The local service map uses 5173 for React, 8080 for Spring Boot, 8000 for FastAPI, 5001 for Node.js, and 3306 for MySQL. The application was verified with these local ports during the final audit.

Shareable project packages should exclude .env files, real passwords, and private credentials. Demo credentials should be distributed separately and rotated when necessary.

9.3 Hosting of Frontend and Backend

The verified project state is intended for local and containerized demonstration. The architecture is suitable for later deployment to a cloud VM or managed service, but this report does not claim an active public production URL. That distinction is important because the final audit verified the application on the local development environment.

For a future production deployment, the React production build can be served by a static host or web server, while the Spring Boot, FastAPI, and Node services can run as separate containers behind a reverse proxy. MySQL can be hosted using a managed relational database service, subject to the deployment requirements of the hosting platform.

GitHub Actions can be extended to build images, run automated tests, publish artifacts, and deploy to a selected hosting environment after project governance and environment secrets are configured.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Frontend blank-page behavior in one browser caused by unsafe assumptions about localStorage user data and a prop mismatch in the search overlay component.
- Missing or incomplete static route configuration for admin, about, help, contact, terms, privacy, and refunds pages.
- Authentication bugs during early iterations where password verification and invalid-login handling were not sufficiently strict.
- Windows-specific Tomcat loopback startup behavior that required a protocol customizer.
- Dependency/build synchronization issues while developing the multiple service stack.
- Need to distinguish course requirements from infrastructure that is present only partially, especially Redis, Kafka, MongoDB, and vector database capabilities.

10.2 Solutions Applied

- Added defensive guards around user state and role handling in the frontend.
- Corrected SearchOverlay close-handler usage and added missing routes.
- Changed authentication so BCrypt hashes are actually verified and invalid login attempts are rejected.
- Added JwtAuthenticationFilter and role-aware authorization enforcement.
- Configured Http11Nio2Protocol to resolve the local Tomcat loopback startup issue.
- Aligned dependency versions and project documentation with the verified runtime stack.
- Performed a final integrity audit covering builds, tests, startup, database connectivity, authentication, RBAC, booking/cancellation, UI, and responsiveness.

10.3 Current Limitations or Known Gaps

The current project uses MySQL as the verified system of record. MongoDB is only PARTIAL in the activity/data path and can fall back to in-memory aggregation in the verified local state.

Redis is classified as PARTIAL because the code/design exists but a live Redis daemon was not verified in the final audit.

Kafka is classified as PARTIAL because producer-side code exists but a live broker/consumer workflow was not verified.

A standalone vector database is NOT IMPLEMENTED. The search feature uses TF-IDF vectors and cosine similarity directly.

The current demonstration is a local/academic deployment rather than a claimed public production system.

Payment processing is represented through the project's payment records and compensation workflow; a live external payment gateway should be treated as a future enhancement unless separately configured and verified.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

1. Introduce a production-ready payment gateway integration with complete webhook verification and reconciliation.
2. Add live Redis infrastructure for caching and session/performance use cases.
3. Complete Kafka broker and consumer integration for event-driven booking/activity flows.
4. Replace the current semantic-search implementation with a dedicated vector database when the project requires large-scale embedding search.
5. Complete MongoDB-backed activity persistence for long-term analytics and audit history.
6. Add stronger observability with centralized metrics, structured traces, and external error monitoring.
7. Extend the organizer portal with richer event analytics, capacity dashboards, and configurable workflows.
8. Introduce email/SMS notifications for booking, cancellation, and event changes.
9. Add automated end-to-end browser tests and broader regression coverage.
10. Prepare a cloud deployment with managed MySQL, container hosting, reverse proxy, and secure secrets management.

11.2 Possible Integrations or Optimizations

A vector database such as pgvector, Qdrant, Weaviate, or Milvus could be introduced when semantic retrieval volume justifies approximate nearest-neighbor indexing. The current system should continue to describe TF-IDF/cosine search honestly until such a database is actually deployed.

Redis can be used to cache frequently requested event summaries and search results while preserving MySQL as the source of truth.

Kafka can decouple activity, notification, and analytics workloads from the synchronous booking transaction path. The booking transaction should remain authoritative for customer-visible consistency, while downstream services consume events asynchronously.

Container orchestration such as Kubernetes could be introduced for high availability, but only after the service contracts, observability, and deployment automation are mature enough to justify the additional complexity.

A mobile client could reuse the existing REST APIs because the business logic and authentication are already separated from the React web layer.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

Universal Tickets - Universal Ticket Booking System was developed as a database-driven full-stack application that brings customer booking, event management, governance, search, and activity services into one integrated platform. The project combines a responsive React frontend, a Spring Boot business backend, a MySQL relational database, a FastAPI gateway/search service, and a Node.js activity service.

The design emphasizes secure access, transaction consistency, clear service boundaries, and maintainable database relationships. The final architecture demonstrates how a relational system of record can coexist with supporting polyglot services without overstating the status of optional infrastructure components.

The implementation also provides a strong demonstration environment for the course because the system contains practical SQL, normalization-oriented schema design, advanced queries, REST APIs, JWT/RBAC security, microservice/service-boundary concepts, Docker/Compose configuration, CI/CD, and semantic search.

12.2 What Was Achieved

- Complete core customer, organizer, verifier, and admin authentication/authorization flow.
- Verified relational database with users, organizer profiles, verifiers, venues, events, bookings, payments, and cancellations.
- Transactional booking and cancellation compensation flow.
- FastAPI semantic search using TF-IDF and cosine similarity.
- Node.js activity-service integration with verified fallback aggregation behavior.
- Frontend build, backend tests, FastAPI tests, activity-service smoke test, startup checks, and responsive UI validation completed successfully in the final audit.
- Project documentation and expo-ready assets aligned with the verified implementation state.

12.3 Skills Learned During Development

- Relational database design, primary/foreign keys, joins, aggregates, CTEs, window functions, and transaction concepts.
- Spring Boot REST API development and JPA/Hibernate persistence.
- FastAPI, Pydantic, OpenAPI, JWT concepts, and semantic search.
- Node.js/Express service design and distributed service boundaries.
- React component design, routing, state handling, responsive UI, and browser debugging.
- Authentication, authorization, BCrypt, JWT access/refresh tokens, and RBAC.
- Docker, Compose, GitHub Actions, testing, documentation, and collaborative Git workflows.

CHAPTER -13 REFERENCES

1. React Documentation - <https://react.dev/>
2. Vite Documentation - <https://vite.dev/>
3. Spring Boot Reference - <https://docs.spring.io/spring-boot/docs/3.2.3/reference/html/>
4. Spring Data JPA Documentation - <https://docs.spring.io/spring-data/jpa/>
5. MySQL 8.0 Reference Manual - <https://dev.mysql.com/doc/refman/8.0/en/>
6. FastAPI Documentation - <https://fastapi.tiangolo.com/>
7. Pydantic Documentation - <https://docs.pydantic.dev/>
8. Express.js Documentation - <https://expressjs.com/>
9. Node.js Documentation - <https://nodejs.org/docs/latest-v20.x/api/>
10. JSON Web Token / JWT project documentation - <https://github.com/jwt/jwt>
11. scikit-learn TF-IDF / cosine similarity documentation - <https://scikit-learn.org/>
12. Docker Documentation - <https://docs.docker.com/>
13. Docker Compose Documentation - <https://docs.docker.com/compose/>
14. GitHub Actions Documentation - <https://docs.github.com/actions>
15. OpenAPI Specification - <https://spec.openapis.org/oas/latest.html>
16. Spring Boot Actuator Documentation - <https://docs.spring.io/spring-boot/reference/actuator.html>
17. OWASP Password Storage Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

The technical details in this report were aligned with the verified project implementation and local audit state rather than with hypothetical integrations. Optional components are identified explicitly as partial or future enhancements where they are not fully live in the current build.

CHAPTER -14 APPENDICES

Appendix A - Application Screenshots

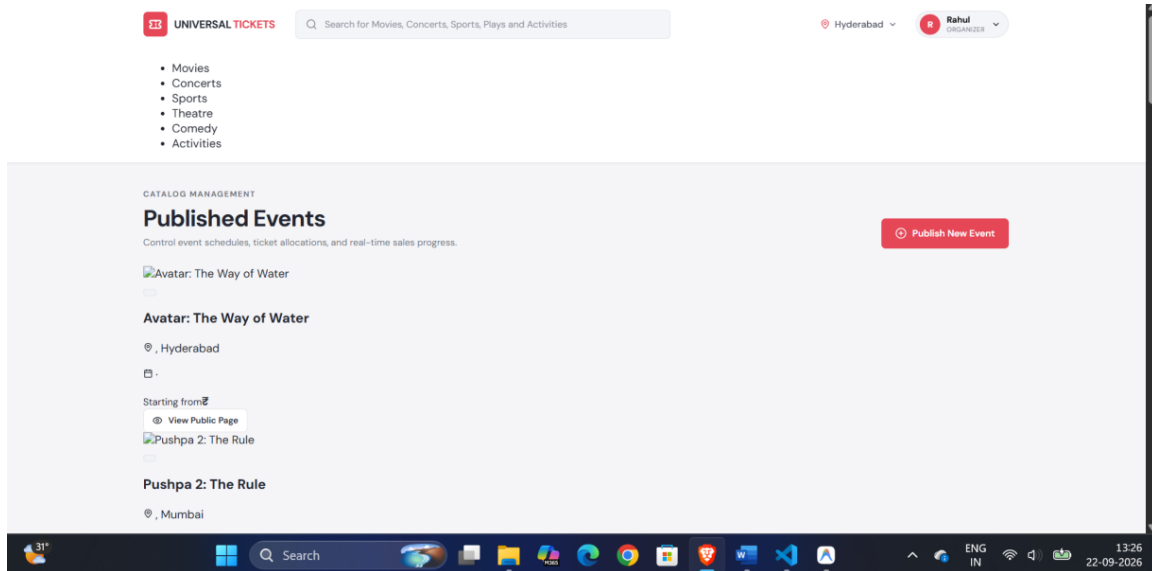


Figure A.1: Published events / organizer or event listing view

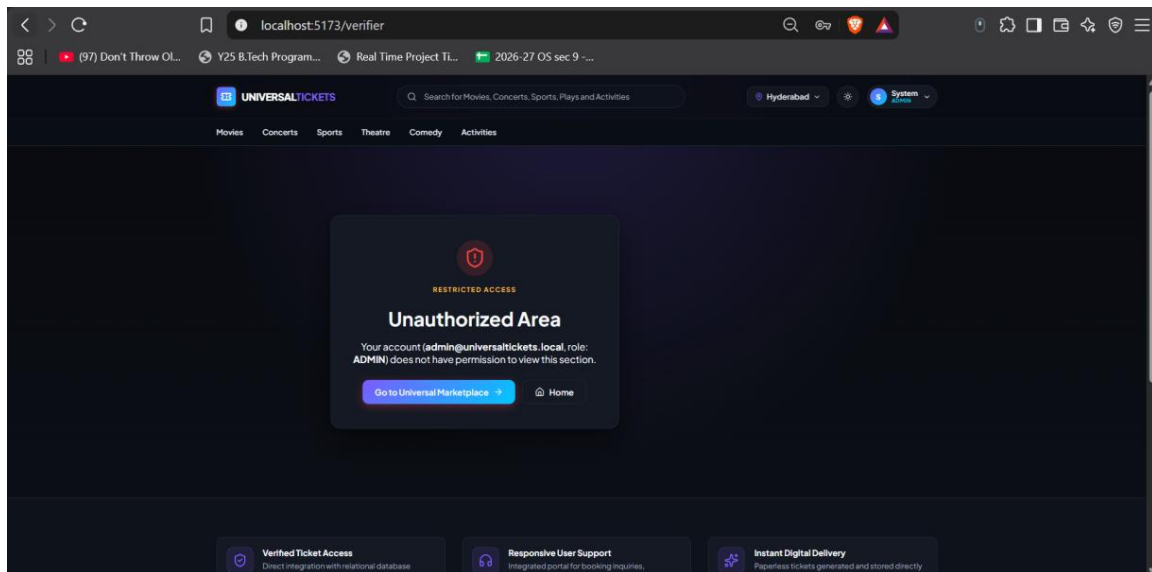


Figure A.2: RBAC unauthorized-access validation

	user_id	full_name
▶	CUS-2026-000001	Rahul Sharma
	CUS-2026-000002	Priya Reddy
	CUS-2026-000003	Arjun Kumar
	CUS-2026-000004	Sneha Rao
	CUS-2026-000005	Vikram Singh
	NULL	NULL

Figure A.3: Users table database snapshot

Appendix B - Database and Architecture Evidence

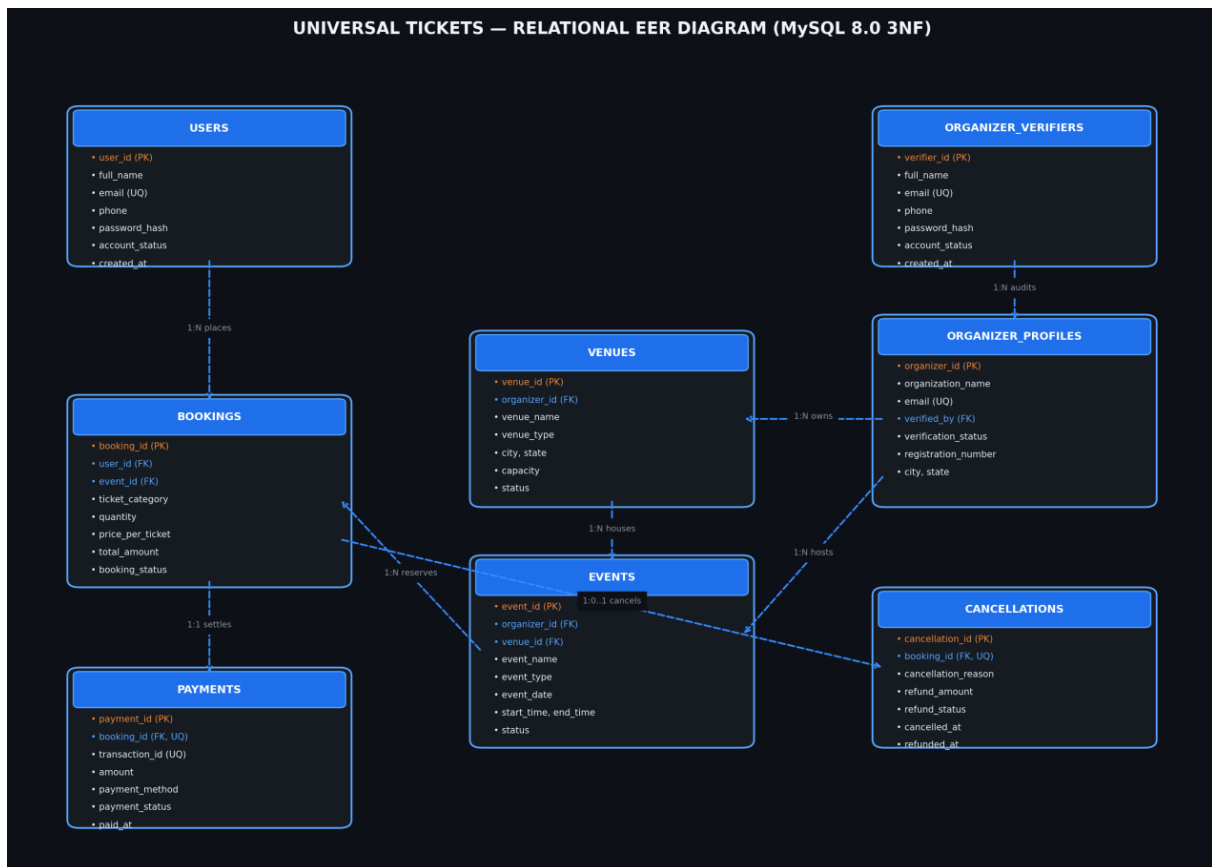


Figure B.1: Final ER diagram

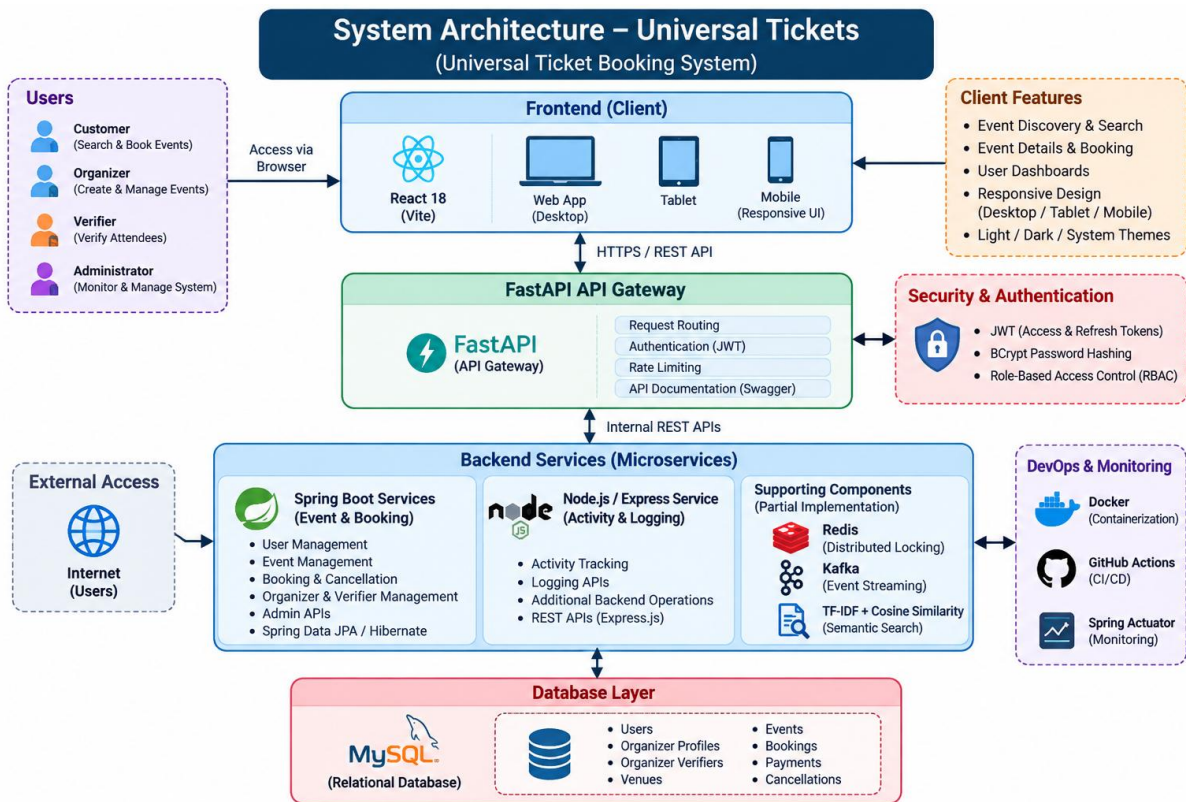


Figure B.2: Final service architecture

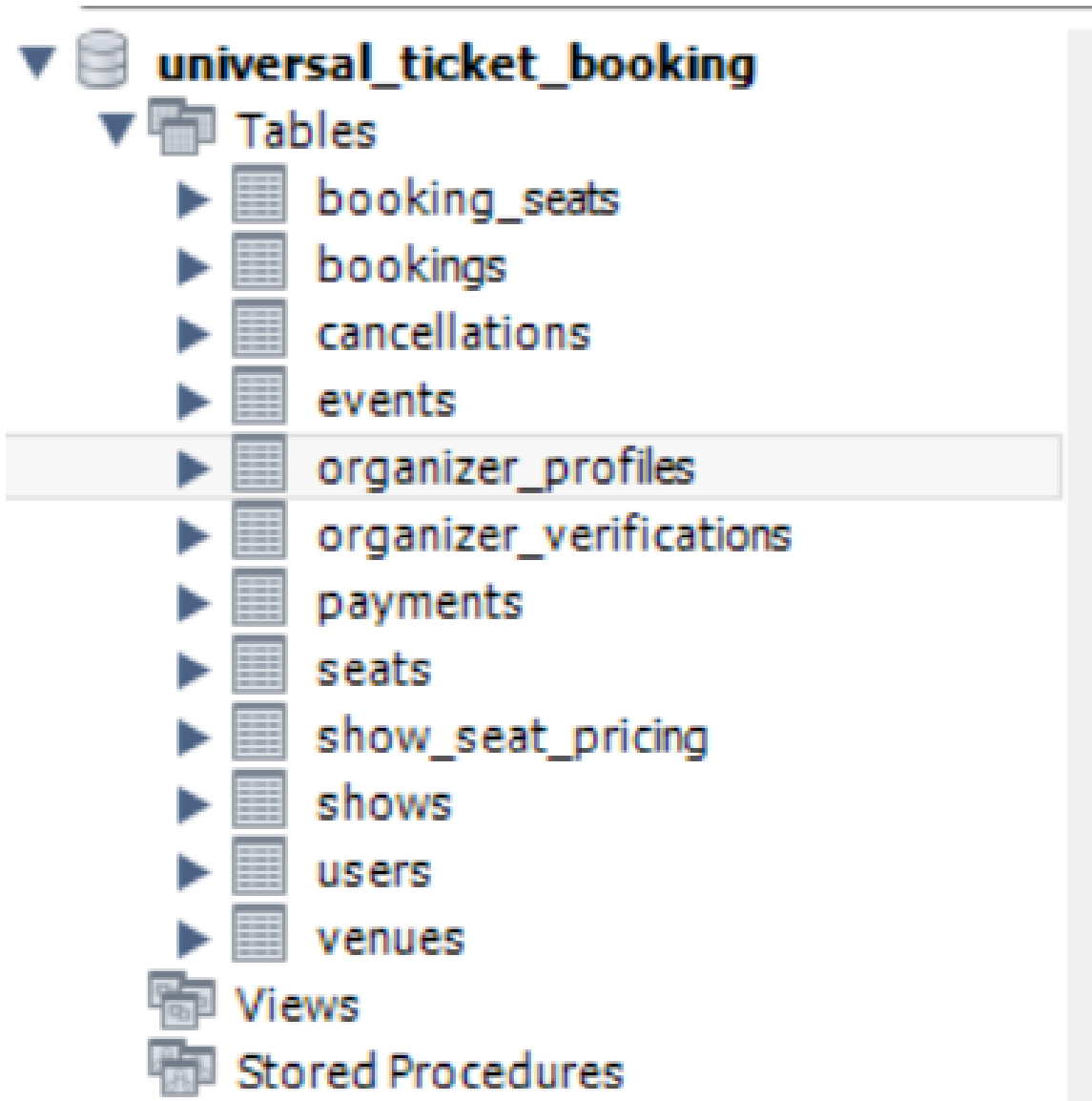


Figure B.3: Database table evidence

Appendix C - Representative Setup Commands

```

Frontend:                                npm                                install
Frontend:                                npm                                run                                dev
Frontend:    build:                        npm                                run                                build
Backend:                                mvn                                clean                                test
Backend:                                mvn                                spring-boot:run
FastAPI:    uvicorn                        main:app                        --reload                        --port                        8000
Node    activity    service:                npm                                install                        &&                                npm                                start
Docker:    docker compose up --build

```

The exact environment variables, local database credentials, and demo passwords should be stored outside the shareable public report. The report documents the architecture and commands without embedding private secrets.

Appendix D - User / Demo Flow

1. Open the React frontend on port 5173.
2. Register or use a prepared local customer account.
3. Log in and verify that customer navigation is displayed.
4. Open event search, search for an event, and inspect event details.
5. Create a booking and verify the booking history entry.
6. Open the cancellation flow and verify that the booking state changes consistently.
7. Demonstrate an admin-only route from a customer account and show the authorization rejection.
8. Log in with the appropriate organizer/verifier/admin account and demonstrate the role-specific portal.
9. Open FastAPI Swagger documentation on port 8000 to demonstrate service-level API documentation.

Appendix E - Final Verified Project Snapshot

Item	Verified State
Frontend build	PASS
Spring Boot build/tests	PASS (3/3 tests)
FastAPI tests	PASS (4/4 tests)
Node activity smoke	PASS
Spring Boot startup	PASS
MySQL	PASS
Authentication / BCrypt	PASS
JWT lifecycle	PASS
RBAC	PASS
Booking + cancellation compensation	PASS
Responsive UI	PASS
Light / Dark / System themes	PASS
Live local dataset snapshot	43 customers, 11 organizers, 30 events, 52 bookings
MongoDB activity persistence	PARTIAL
Redis	PARTIAL
Kafka	PARTIAL
Standalone vector database	NOT IMPLEMENTED